

ShopFlow Curriculum · Part D — Data Structures & Algorithms (Integrated Edition)

Part D — Data Structures & Algorithms — Integrated Edition

Run in parallel with Parts A-C. Interview-complete DSA, rebuilt so it never sits in isolation: a week-by-week schedule that starts inside Part A, a Big-O / math / pattern-recognition foundation chapter, and a Theory → Java → LeetCode → ShopFlow pipeline for every one of the 17 topics below.

17 topics (16 core + 1 optional) · 85 hands-on projects · ~163 LeetCode problems · 13 weeks, concurrent with Parts A-C

What This Revision Changes

- DSA is no longer isolated. A new weekly parallel schedule (below) starts DSA in Week 1 of Part A — Backend Fundamentals — instead of waiting for Parts A-C to finish first.
- Five new topics added: Recursion (9.0, taught first), Backtracking (9.7), Tries (9.10, promoted out of a footnote), Greedy Algorithms (9.13), and Bit Manipulation (9.14).
- Reordered: Sorting & Searching now precedes Trees/Graphs/DP — binary search is a prerequisite mental model, not just its own topic. Hashing moved up to sit right after Arrays.
- New Foundations chapter covering Big-O notation, the math underneath it, a problem-solving framework (UMPIRE), a debugging-algorithms checklist, a pattern-recognition lookup table, a Blind 75 / NeetCode 150 crosswalk, and a time/space optimization checklist.
- Every topic now carries a Big-O panel (not just a suggested-projects list) and an explicit ShopFlow Application callout, so DS theory, Java implementation, LeetCode practice, and the running project are never four disconnected activities.
- Union-Find elevated into its own focused topic (9.12); Segment/Fenwick Trees kept as an explicitly optional appendix (9.16) for quant/HFT-track readers only.

How This Fits Together

Why DSA runs beside Parts A-C instead of after them

The single biggest structural problem with treating DSA as “Part D, done after Parts A-C” is that it front-loads a year of backend/frontend/deployment work before any interview-pattern practice begins — and DSA fluency decays fast without repetition. This edition instead treats DSA as a constant weekly habit that runs underneath whichever part you’re actively building, the same way a gym routine runs underneath a work schedule rather than waiting for a vacation.

The Rule — Parallel, Not Sequential. Every week, regardless of which Part you are in, reserve 8-12 hours split roughly 60% DSA (theory + from-scratch build + LeetCode set) / 40% whatever Part A/B/C project work is active that week. DSA topics are sequenced so each one’s Java implementation or ShopFlow application naturally lines up with what that week’s Part A/B/C work already has you touching.

Week	Concurrent With	DSA Topic(s)	Why They Pair
1-2	Part A — Backend Fundamentals (Java syntax, OOP review)	9.0 Recursion	Needs nothing but a compiler — starts day one, before Spring Boot exists
1-3	Part A — Backend Fundamentals (collections, DTOs)	9.1 Arrays & Strings	You’re already slicing lists in controllers; this names the pattern
3	Part A — Backend Fundamentals (JPA repositories, indexing)	9.2 Hashing	HashMap is the in-memory analogue of the DB index you’re about to build
4	Part A — Backend Fundamentals (pagination, query params)	9.3 Two Pointers & Sliding Window	Cursor-based pagination is a sliding window over results
5	Part A → Part B transition (React state, events)	9.4 Stacks & Queues	The call stack and render queue you’re now debugging in the browser
6	Part B — Frontend Fundamentals (sortable tables)	9.5 Sorting & Searching	You’re wiring a ‘sort by price’ dropdown — know what’s underneath it
7	Part B — Frontend Fundamentals (component trees, state)	9.6 Linked Lists	Immutable state updates mirror functional linked-list operations
8	Part B — Frontend Fundamentals (multi-step forms, filters)	9.7 Backtracking	Filter-combination UIs are a smaller cousin of subsets/combinations
9	Part B → Part C transition (nested menus, config trees)	9.8 Trees	ShopFlow’s category navigation IS a tree — build feature and topic together
10	Part C — Deployment & Cloud (job queues, search infra)	9.9 Heaps · 9.10 Tries	Background-job priority queues and the search-bar autocomplete
11	Part C — Deployment & Cloud (service graphs, routing)	9.11 Graphs · 9.12 Union-Find	Microservice call graphs and CDN routing are graph problems
12	Part C — Deployment & Cloud (autoscaling, feature flags)	9.13 Greedy · 9.14 Bit Manipulation	Autoscaler thresholds are greedy; role flags are bitmasks
13	Part C wrap-up → Part D full-time	9.15 Dynamic Programming	The one topic worth a dedicated, full-

Cadence stays 8–12 hrs/week · 2–3 DSA sessions/week, matching the original pace — only the sequencing against Parts A–C is new.

Interview Foundations

Read once before Week 1 — referenced constantly after.

1 · Big-O, Properly

Big-O describes how an algorithm’s cost grows as input size n grows large — it is a statement about the shape of growth, not a stopwatch reading. Drop constants ($O(2n) \rightarrow O(n)$) and lower-order terms ($O(n^2 + n) \rightarrow O(n^2)$) because they stop mattering as $n \rightarrow \infty$. When combining costs: sequential steps add ($O(n)$ then $O(m) \rightarrow O(n + m)$); nested steps multiply (a loop of n inside a loop of $m \rightarrow O(n \cdot m)$).

Complexity	Name	Example
$O(1)$	Constant	array index, HashMap get
$O(\log n)$	Logarithmic	binary search, balanced BST ops
$O(n)$	Linear	single pass over an array
$O(n \log n)$	Linearithmic	merge sort, heap sort, sort-then-scan
$O(n^2)$	Quadratic	nested loops, naive pair-checking
$O(2^n)$	Exponential	naive recursive Fibonacci, subset generation
$O(n!)$	Factorial	generating all permutations

2 · Math Foundations You Actually Use

- Logarithms:** $\log_2 n$ = “how many times can you halve n before reaching 1” — this is the entire reason binary search, balanced trees, and heaps are $O(\log n)$: each step eliminates half the remaining possibilities.
- Summations & series:** $1+2+\dots+n = n(n+1)/2 = O(n^2)$ explains why a naive nested-pair loop is quadratic even though the inner loop shrinks each time.
- Combinatorics:** 2^n subsets, $n!$ permutations, $C(n,k)$ combinations — these are not just trivia, they are the time complexity of Backtracking (9.7) and Bit Manipulation (9.14) problems; recognizing which counting formula applies tells you the complexity before you write code.
- Recurrence relations:** $T(n) = 2T(n/2) + O(n)$ is merge sort’s recurrence, solving (via the Master Theorem, informally: “work per level $\times$ number of levels”) to $O(n \log n)$ — the same reasoning explains why naive recursive Fibonacci’s $T(n) = T(n-1) + T(n-2)$ is $O(2^n)$.
- Modular arithmetic:** shows up directly in Bit Manipulation (9.14) and in hashing (9.2) — bucket index = $\text{hash}(\text{key}) \bmod \text{tableSize}$.

3 · The UMPIRE Problem-Solving Framework

Use this on every LeetCode problem and every interview question, out loud: **U**nderstand the problem (restate it, ask about edge cases and constraints) $\rightarrow$ **M**atch it to a pattern (scan the Pattern Recognition table below) $\rightarrow$ **P**lan the approach in plain English or pseudocode before touching a keyboard $\rightarrow$ **I**mplement it $\rightarrow$ **R**eview by tracing a small example by hand $\rightarrow$ **E**valuate the final time/space complexity out loud. Skipping straight from Understand to Implement is the single most common reason a correct-idea solution runs out of time in an interview.

4 · Pattern Recognition — What the Problem Statement Is Telling You

Signal in the problem	Likely pattern	Topic
Sorted array / linked list	Two pointers, binary search	9.1 / 9.3 / 9.5
“Contiguous subarray/substring”	Sliding window, Kadane’s	9.1 / 9.3
“Top K”, “K closest”, “K largest”	Heap of size K	9.9
“All subsets / permutations / combinations”	Backtracking, bitmask	9.7 / 9.14
Prefix search / autocomplete	Trie	9.10
Grid or graph, “shortest path”, “reachable”	BFS/DFS, Dijkstra	9.11
“Group / merge connected items”	Union-Find	9.12
“Prerequisites”, “order of tasks”	Topological sort	9.11
“Maximize/minimize” with local choices, sorted input	Greedy — verify greedy-choice property first	9.13
“Count the ways”, “min/max cost to reach”, overlapping subcases	Dynamic programming	9.15
Recursive structure not obviously a tree/graph	Recursion $\rightarrow$ check for memoizable overlap	9.0 / 9.15

5 · Debugging Algorithms Systematically

- **Trace on paper first.** Run your algorithm by hand on the smallest input that could possibly fail ($n=0$, $n=1$, $n=2$) before adding a single print statement.
- **Binary-search the bug.** For a long pipeline (e.g. build graph $\rightarrow$ BFS $\rightarrow$ reconstruct path), print/assert intermediate state at the midpoint to halve the search space for where it broke — same idea as binary search, applied to your own code.
- **Off-by-one checklist:** loop bound ($<$ vs. $\leq$), window boundaries in sliding window, recursion base case, and array index math ($i-1$ vs. i) are the four most common sources.
- **State-mutation checklist (backtracking specifically):** did you undo every choice you made before trying the next branch? A missing un-choose step is the most common backtracking bug.
- **Compare against brute force.** For any optimized solution, keep the $O(n^2)$ -or-worse brute force around and diff outputs on random small inputs — this catches logic bugs the eye misses.

6 · Time & Space Optimization Checklist

- Can a hash map remove a nested loop? Trading $O(n)$ space for a lookup often turns $O(n^2)$ into $O(n)$.
- Can sorting first simplify the logic? $O(n \log n)$ sort + linear scan often beats an unsorted-data $O(n^2)$ approach.
- Can you go one pass instead of two? Track a running min/max/sum instead of a first pass to compute it and a second to use it.
- Does the DP table need all its history? If $dp[i]$ only depends on $dp[i-1]$ and $dp[i-2]$ (House Robber, Fibonacci), roll it down from $O(n)$ to $O(1)$ space — same idea flattens 2D knapsack/LCS tables to $O(m)$.
- Is recursion adding hidden $O(n)$ stack space a plain loop wouldn't need? Worth naming out loud even if you keep the recursive version for clarity.

7 · Blind 75 / NeetCode 150 Crosswalk

Both lists are curated problem sets, not separate curricula — every category in each maps onto a topic already in this phase. Work the suggested LeetCode volume on each topic page; if you want an external checklist to cross-reference, use this mapping rather than treating Blind 75 as an eighteenth topic to schedule separately.

Blind 75 / NeetCode 150 category	Maps to Topic(s)
Arrays & Hashing	9.1 Arrays & Strings, 9.2 Hashing
Two Pointers	9.3 Two Pointers & Sliding Window
Sliding Window	9.3 Two Pointers & Sliding Window
Stack	9.4 Stacks & Queues
Binary Search	9.5 Sorting & Searching
Linked List	9.6 Linked Lists
Trees	9.8 Trees
Tries	9.10 Tries
Heap / Priority Queue	9.9 Heaps & Priority Queues
Backtracking	9.7 Backtracking
Graphs / Advanced Graphs	9.11 Graphs, 9.12 Union-Find
1-D DP / 2-D DP	9.15 Dynamic Programming
Greedy Intervals	9.13 Greedy Algorithms
Bit Manipulation	9.14 Bit Manipulation
Math & Geometry	9.0 Recursion (recursive math problems)

9.0 · Recursion & Recursive Thinking NEW

The mental model behind Trees, Backtracking, and DP — taught first, not absorbed by accident.

Subtopics: base case & recursive case · the call stack, visualized · recursion trees · converting recursion to iteration · tail vs. non-tail recursion

PARALLEL WEEK. Weeks 1–2, parallel with Part A · Backend Fundamentals (Java syntax, OOP review). Recursion needs nothing but a compiler, so it starts on day one — do not wait for Spring Boot to “finish” first.

Theory. Every recursive function is a proof by mathematical induction that happens to execute: the base case is the base of the induction, the recursive case is the inductive step. The call stack is the mechanism, not the concept — draw the recursion tree before writing code, and you can read the time complexity directly off its shape (branching factor and depth). Tail recursion is what a compiler could turn into a loop; Java's JVM does not do this optimization, which is exactly why the iteration-conversion drill matters for interviews, not just theory.

Complexity at a Glance

Operation	Time	Space
Linear recursion (n calls)	O(n)	O(n) call stack
Binary recursion, no memo (e.g. naive Fibonacci)	O(2^n)	O(n) call stack
Divide & conquer, halving (e.g. fast power)	O(log n)	O(log n) call stack
Tail-recursive version, converted to loop	O(n)	O(1)

Java Notes. Model each call’s local state explicitly before coding: parameters + what changes each call. Use -Xss JVM flag knowledge as trivia only — in interviews, assume unbounded recursion on large input is a red flag worth mentioning out loud.

Interview patterns: Induction / base-case identification · Divide & conquer · State-space reduction

Hands-On Projects

Factorial & Fast Power — Recursive factorial(n) → power(base, exp) via fast exponentiation (halve exp each call) → trace the call stack on paper for n=5 → convert both to iterative and compare space.

Sum of Digits & Digit Reversal — Recursive sumOfDigits(n) → recursive reverseNumber(n) → test with negative and single-digit input → identify the exact base case that prevents infinite recursion.

Recursive Array Search & Sum — Recursive linearSearch(arr,target,index) → arraySum(arr,index) → arrayMax(arr,index) → test on empty and single-element arrays.

Tower of Hanoi — 3-peg recursive solution printing each move → confirm move count = 2^n – 1 → explain why it’s awkward iteratively → time n=20 and discuss growth.

Recursion → Iteration Drill — Convert 3 recursive solutions above to iterative using an explicit stack where needed → compare space before/after → note which direction felt natural.

SHOPFLOW APPLICATION. Recursion is the mechanism you’ll use in 9.8 to walk ShopFlow’s category tree (Electronics → Laptops → Gaming Laptops) into a nested menu — do the Tower-of-Hanoi/array drills here so the category-tree recursion in Trees is implementation, not a new concept.

Suggested LeetCode volume: 8 problems (mostly Easy/Medium)

Study Resources

- GeeksforGeeks — Recursion in Java — [geeksforgeeks.org/java/recursion-in-java](https://www.geeksforgeeks.org/java/recursion-in-java)
- NeetCode — Roadmap — neetcode.io/roadmap

9.1 · Arrays & Strings

The most fundamental structure, and the pattern toolkit that solves a huge share of interview questions.

Subtopics: two pointers · sliding window · prefix sums · Kadane’s algorithm (max subarray) · StringBuilder & string manipulation

PARALLEL WEEK. Weeks 1–3, parallel with Part A · Backend Fundamentals (Java collections, request/response DTOs). You’re already slicing arrays and lists when you build REST controllers — this topic names the patterns you’re using informally.

Theory. Arrays are the substrate almost every other structure is built on top of (a hash table is an array + a hash function; a heap is an array with an implicit tree shape). Two pointers and sliding windows both exploit monotonicity — some property that only moves in one direction as a pointer advances — to turn an O(n²) brute force into O(n). Kadane’s algorithm is the smallest example of the DP idea (“the best answer ending here”) you’ll formalize in 9.15.

Complexity at a Glance

Operation	Time	Space
Two-pointer scan	O(n)	O(1)
Sliding window (variable size)	O(n)	O(1)–O(k)
Prefix sum build + O(1) range query	O(n) build / O(1) query	O(n)
Kadane’s algorithm	O(n)	O(1)
Naive string concatenation in a loop	O(n²)	O(n)
StringBuilder append in a loop	O(n)	O(n)

Java Notes. Prefer StringBuilder over String concatenation in any loop — Strings are immutable, so s += c is a hidden O(n) copy every iteration.

Interview patterns: Two pointers · Sliding window · Prefix sum · Kadane’s / running-best DP

Hands-On Projects

Two Sum (and variants) — Brute-force O(n²) → HashMap O(n) → sorted-array two-pointer version → compare all three.

Longest Substring Without Repeating Characters — Window with two pointers + a Set → track max length as window grows/shrinks → test empty string, all-same characters.

Prefix Sum Range Queries — Build prefix sum array → answer ‘sum i..j’ in O(1) → compare to recomputation → apply to ‘subarray sums to target’.

Kadane’s Algorithm — Track running sum (reset when negative) and running max separately → return subarray start/end too → test all-negative array.

String Reversal & Palindrome Check — Reverse with StringBuilder, then two pointers in place → palindrome check with two pointers → handle spaces/punctuation/case.

Merge Intervals — Sort by start time → merge overlaps in one pass → return minimal list → test intervals fully contained in others.

SHOPFLOW APPLICATION. Kadane’s / prefix sums are exactly how you’d compute ‘best 7-day revenue window’ for a seller dashboard; Merge Intervals is the algorithm behind resolving overlapping promo/coupon date ranges (reused explicitly in 9.13).

Suggested LeetCode volume: 15 problems (5 Easy / 8 Medium / 2 Hard)

Study Resources

- GeeksforGeeks — Array Data Structure
- NeetCode — Arrays & Hashing
- Oracle — Arrays (Java Tutorials)

9.2 · Hashing

Trading memory for near-constant-time lookups — sits right after Arrays, where it’s used constantly.

Subtopics: HashMap internals (buckets & hashing) · collision handling · frequency maps · the two-sum family

PARALLEL WEEK. Week 3, parallel with Part A · Backend Fundamentals (JPA repositories, indexing). A HashMap is the in-memory analogue of the DB index you’ll build in Part A — understanding buckets/collisions makes the index conversation with a DBA make sense later.

Theory. A hash table’s $O(1)$ is an average-case amortized guarantee, not a hard one — it depends on a good hash function spreading keys evenly across buckets and a load factor that triggers resizing before chains get long. Knowing why worst case degrades to $O(n)$ (all keys colliding into one bucket) is what separates a memorized answer from an understood one in an interview follow-up.

Complexity at a Glance

Operation	Time	Space
get / put / contains	$O(1)$	$O(n)$
Iteration over all entries	$O(n)$	$O(n)$
Frequency map build	$O(n)$	$O(n)$

Java Notes. Java’s HashMap resizes at load factor 0.75 and (since Java 8) treeifies buckets with 8+ collisions into a red-black tree, capping worst-case at $O(\log n)$ — a good detail to mention if asked about worst case.

Interview patterns: Hash lookup instead of nested loop · Frequency counting · Complement search (two-sum family)

Hands-On Projects

Implement a Basic HashMap — Array of buckets → hashCode-based bucket selection → chaining for collisions → test intentionally colliding keys.

Frequency Map Drill — Character frequency in a string → word frequency in a sentence → most frequent element → ‘first non-repeating character’.

Two Sum Family, Revisited — Two-sum with HashMap in one pass → three-sum by fixing one element and hashing the rest → compare to brute force.

Group Anagrams — Group words into anagram sets using sorted-characters string as key → return List<List> → test words with no anagrams.

Longest Consecutive Sequence — All numbers into a HashSet → check if each is a sequence start → extend forward counting → confirm $O(n)$ beats sort-based $O(n \log n)$.

SHOPFLOW APPLICATION. Frequency maps power ‘trending searches’ on ShopFlow’s homepage; a HashMap of SKU → inventory record is the $O(1)$ lookup your cart and checkout services depend on before anything hits PostgreSQL.

Suggested LeetCode volume: 10 problems (4 Easy / 5 Medium / 1 Hard)

Study Resources

- GeeksforGeeks — Hashing Data Structure
- Oracle — HashMap (Javadoc)
- NeetCode — Arrays & Hashing

9.3 · Two Pointers & Sliding Window — Pattern Review NEW

Pulled out of Arrays into its own explicit review so the pattern gets named, not just used once and forgotten.

Subtopics: fixed vs. variable window · shrink/grow conditions · monotonic deque (sliding window maximum) · when two pointers beats a window

PARALLEL WEEK. Week 4, parallel with Part A · Backend Fundamentals (pagination & query params). Cursor-based pagination on a product-list endpoint is a sliding window over a result set — build both at once.

Theory. The recurring interview question is ‘two pointers or sliding window?’ — the honest answer is that a fixed/variable-size contiguous window is a two-pointer technique; the distinction that matters is whether the window’s validity condition is monotonic (only gets better or only gets worse as you grow it). If it isn’t monotonic, neither pattern applies and you likely need DP.

Complexity at a Glance

Operation	Time	Space
Sliding window max (monotonic deque)	O(n)	O(k)
Variable window (min window substring)	O(n)	O(k)
Fixed window, naive re-scan	O(n·k)	O(1)
Floyd’s cycle detection (fast/slow ptr)	O(n)	O(1)

Java Notes. A monotonic deque needs ArrayDeque, not Stack or LinkedList directly — it supports O(1) push/pop from both ends.

Interview patterns: Fixed window · Variable/shrinking window · Monotonic deque · Fast/slow pointers

Hands-On Projects

Sliding Window Maximum — Monotonic decreasing deque of indices → pop back while new element larger → pop front once index leaves window → front = max at each step.

Minimum Window Substring — Expand until valid (contains all required chars) → shrink from left while still valid, tracking minimum → frequency map for validity → test target longer than any valid window.

Fixed-Size Window Drill — ‘Max sum subarray of size k’ → ‘first negative number in every window of size k’ → compare naive O(n·k) to O(n) sliding version.

Two Pointers vs. Sliding Window — ‘Container with most water’ via two pointers → ‘longest substring with ≤K distinct chars’ via window → write down the signal that tells you which to reach for.

Find the Duplicate Number — Treat array values as linked-list ‘next’ pointers → Floyd’s cycle detection finds the duplicate → confirm no extra space or array mutation.

SHOPFLOW APPLICATION. This is literally your product-search-results pagination and the ‘infinite scroll’ feed — a fixed window of page size sliding over a sorted result cursor.

Suggested LeetCode volume: 8 problems (3 Easy / 5 Medium)

Study Resources

- [GeeksforGeeks — Sliding Window Technique](#)
- [NeetCode — Sliding Window](#)

9.4 · Stacks & Queues

LIFO and FIFO structures underlying everything from undo buttons to breadth-first search.

Subtopics: stack (push/pop/peek) · queue (enqueue/dequeue) · min-stack · monotonic stack · implementing both from scratch

PARALLEL WEEK. Week 5, parallel with Part A → Part B transition (React state, event handling). The browser’s call stack and React’s render queue are the exact structures you’re implementing from scratch here.

Theory. A monotonic stack is the array analogue of a monotonic sliding window: you maintain an invariant (increasing or decreasing) by popping elements that violate it, and — the detail people miss — every element is pushed and popped at most once, which is why an apparently nested loop is still O(n) total, not O(n²).

Complexity at a Glance

Operation	Time	Space
push / pop / peek (array or linked list)	O(1)	O(n) total
enqueue / dequeue	O(1)	O(n) total
getMin() with auxiliary min-stack	O(1)	O(n) total
Next Greater Element (monotonic stack)	O(n) total	O(n)

Java Notes. Do not use java.util.Stack (it’s a legacy, synchronized Vector subclass); use ArrayDeque as a stack via push()/pop(), and as a queue via offer()/poll().

Interview patterns: Matching / validity via stack · Monotonic stack · Two-stack queue simulation

Hands-On Projects

Stack From Scratch — push/pop/peek/isEmpty backed by resizable array → clear exception on empty-pop → confirm LIFO order.

Valid Parentheses — Stack matches {} → return false on mismatch or leftover open bracket → test nested and malformed input.

Min-Stack — Second stack tracks running minimum → push to min-stack only on new minimum → pop both together → test duplicate minimums.

Queue From Scratch — enqueue/dequeue/peek/isEmpty → justify array vs linked-list backing in a comment → confirm FIFO order.

Monotonic Stack: Next Greater Element — O(n) with monotonic decreasing stack → compare to brute-force O(n²) → test strictly decreasing input.

Queue Using Two Stacks — enqueue/dequeue with only two Stack instances → identify which op moves elements → confirm amortized cost stays reasonable.

SHOPFLOW APPLICATION. A monotonic-stack style undo/redo stack backs the admin product-editor’s ‘undo last change’; a request queue is how you’d throttle outbound emails from the order-confirmation service.

Suggested LeetCode volume: 10 problems (3 Easy / 6 Medium / 1 Hard)

Study Resources

- GeeksforGeeks — Stack Data Structure
- GeeksforGeeks — Queue Data Structure
- NeetCode — Stack

9.5 · Sorting & Searching

Binary search variants are a prerequisite mental model for the Graph and DP work ahead, not just an interview topic on their own.

Subtopics: merge sort · quick sort · binary search & its variants · time complexity trade-offs

PARALLEL WEEK. Week 6, parallel with Part B · Frontend Fundamentals (sortable/filterable product tables). You’re wiring a ‘sort by price’ dropdown in React — know what’s happening underneath it.

Theory. Merge sort’s guaranteed $O(n \log n)$ comes at the cost of $O(n)$ auxiliary space and is stable; quick sort is in-place but its worst case (already-sorted input with a naive pivot) degrades to $O(n^2)$ — this trade-off, not the code itself, is the actual interview question. Binary search’s real skill isn’t the ‘find exact match’ version; it’s recognizing when a problem has a monotonic yes/no answer over a range (“can I finish in X days?”) and binary-searching the answer space itself.

Complexity at a Glance

Operation	Time	Space
Merge sort	$O(n \log n)$ all cases	$O(n)$
Quick sort (average / worst)	$O(n \log n)$ / $O(n^2)$	$O(\log n)$
Binary search	$O(\log n)$	$O(1)$
Search in rotated sorted array	$O(\log n)$	$O(1)$

Java Notes. `Arrays.sort()` on primitives is a dual-pivot quicksort ($O(n \log n)$ avg, $O(n^2)$ worst); on objects it’s a stable Timsort ($O(n \log n)$ guaranteed) — know which one you get.

Interview patterns: Divide & conquer sort · Binary search on index · Binary search on answer space

Hands-On Projects

Merge Sort From Scratch — Recursive divide step → merge step combining sorted halves → test sorted and reverse-sorted input → confirm stability.

Quick Sort From Scratch — Partition step (Lomuto or Hoare) → recursive quicksort around it → test worst case on sorted array → discuss pivot choice.

Binary Search & Its Variants — Standard exact-match search → ‘find first occurrence’ with duplicates → ‘find insertion point’ → test empty array, single element.

Search in a Rotated Sorted Array — Find target in $O(\log n)$ with unknown pivot → determine which half is properly sorted at each step → test target = pivot element.

Sorting Algorithm Comparison — Time your merge sort and quick sort on the same large random array → time `Arrays.sort()` → explain any difference → note pick for nearly-sorted data.

SHOPFLOW APPLICATION. Binary-search-on-answer-space is exactly how you’d implement ‘cheapest shipping option that arrives by date X’; product list sorting (price/rating/newest) is this topic, live, in the UI.

Suggested LeetCode volume: 10 problems (3 Easy / 6 Medium / 1 Hard)

Study Resources

- GeeksforGeeks — Sorting Algorithms
- GeeksforGeeks — Binary Search
- NeetCode — Binary Search

9.6 · Linked Lists

A chain of nodes where insertion/deletion is cheap but random access isn’t — the classic source of pointer-manipulation questions.

Subtopics: singly & doubly linked lists · fast/slow pointer technique · reversing a list · cycle detection

PARALLEL WEEK. Week 7, parallel with Part B · Frontend Fundamentals (component trees, state updates). Immutable state updates in React ('return a new node chain, don't mutate') are the same discipline as functional linked-list operations.

Theory. A linked list trades array's $O(1)$ random access for $O(1)$ insertion/deletion anywhere you already hold a pointer — the entire topic is pointer bookkeeping under that trade. The LRU Cache project is the payoff: it's the textbook demonstration that combining two structures (HashMap for $O(1)$ lookup + doubly linked list for $O(1)$ reordering) can beat what either does alone.

Complexity at a Glance

Operation	Time	Space
Insert/delete at head	$O(1)$	$O(1)$
Insert/delete at tail (singly, no tail ptr)	$O(n)$	$O(1)$
Search / get(index)	$O(n)$	$O(1)$
Reverse (iterative)	$O(n)$	$O(1)$
LRU Cache get/put	$O(1)$	$O(\text{capacity})$

Java Notes. Prefer manual Node classes over `java.util.LinkedList` while learning — using the built-in defeats the point of the exercise, though it's the right production choice later.

Interview patterns: Fast/slow (Floyd's) pointers · Dummy head node · In-place reversal

Hands-On Projects

Build a Singly Linked List From Scratch — Node class (value, next) → add/remove/get(index) → toString() for debugging → test insertion at head, middle, tail.

Reverse a Linked List — Iterative 3-pointer (prev/curr/next) → recursive version → compare space complexity → test empty and 1-node list.

Detect a Cycle (Floyd's Algorithm) — Build a list with a deliberate cycle → fast/slow pointers detect it → extend to find where the cycle begins → confirm no false positive on acyclic list.

Doubly Linked List Implementation — Extend Node with prev → $O(1)$ insertion/removal from both ends → forward/backward traversal → base for LRU Cache.

Merge Two Sorted Linked Lists — Merge without creating new nodes, just relinking → test lists of different lengths → test one empty list.

LRU Cache — HashMap ($O(1)$ lookup) + doubly linked list ($O(1)$ reorder) → get() moves node to front → put() evicts tail when over capacity → test eviction order at capacity 2.

SHOPFLOW APPLICATION. The LRU Cache project is ShopFlow's product-detail page cache — cap it at N recently-viewed products and it directly becomes the 'Recently Viewed' rail.

Suggested LeetCode volume: 10 problems (3 Easy / 6 Medium / 1 Hard)

Study Resources

- GeeksforGeeks — Linked List Data Structure
- Baeldung — Introduction to Linked Lists in Java
- NeetCode — Linked List

9.7 · Backtracking NEW

One of the highest-yield patterns for new-grad/internship loops specifically.

Subtopics: the choose/explore/un-choose template · pruning · subsets vs. permutations vs. combinations · constraint satisfaction

PARALLEL WEEK. Week 8, parallel with Part B · Frontend Fundamentals (multi-step forms, filter combinations). Building a product-filter UI (brand × size × color combinations) is a smaller cousin of the subsets/combinations you'll write here.

Theory. Every backtracking solution is the same three-line template — choose, explore, un-choose — applied to a different constraint. The un-choose step (undoing a decision before trying the next branch) is what people forget and it's the single most common bug source: mutate shared state, forget to revert it, and every branch after the first sees corrupted state. Pruning is what makes the exponential search tree tractable in practice even though its worst-case complexity doesn't change.

Complexity at a Glance

Operation	Time	Space
Subsets (n elements)	$O(2^n \cdot n)$	$O(n)$ recursion depth
Permutations (n elements)	$O(n! \cdot n)$	$O(n)$ recursion depth
N-Queens (n queens)	$O(n!)$ worst, pruned in practice	$O(n)$
Word Search (m×n grid)	$O(m \cdot n \cdot 4^L)$	$O(L)$

Java Notes. Use a single mutable List/array + explicit backtrack (add, recurse, remove) rather than copying collections at each call — copying turns an already-exponential algorithm needlessly slower.

Interview patterns: Choose / explore / un-choose · Constraint pruning · DFS over decision tree

Hands-On Projects

Subsets — Choose/explore/un-choose template → all subsets, no duplicates → extend for duplicate elements without duplicate subsets → confirm count = 2^n.

Permutations — Swap-based backtracking → 'used[]' boolean-array approach → compare clarity/space → handle duplicates without duplicate permutations.

Combination Sum — Combinations summing to target, reusing numbers → variant using each number once → prune once running sum exceeds target → test no valid combination.

N-Queens — Place queens row by row checking column/diagonal conflicts → sets or bitmasks for O(1) conflict checking → count solutions for n=8 → print one valid board.

Word Search — DFS from each cell matching next char → mark visited on path, un-mark on backtrack → prune early → test a word wrapping the grid edge.

SHOPFLOW APPLICATION. This is the exact engine behind 'generate valid bundle offers under a price constraint' — choose products, explore combinations, prune any branch that exceeds budget, un-choose and continue.

Suggested LeetCode volume: 10 problems (1 Easy / 7 Medium / 2 Hard)

Study Resources

- GeeksforGeeks — Backtracking Algorithms
- GeeksforGeeks — Introduction to Backtracking
- NeetCode — Backtracking

9.8 · Trees

Hierarchical, recursive structures — the shape behind file systems, category menus, and JSON itself.

Subtopics: binary tree traversals (in/pre/post/BFS) · binary search tree operations · tree height & balance · lowest common ancestor

PARALLEL WEEK. Week 9, parallel with Part B → Part C transition (nested UI menus, config trees). ShopFlow's category navigation IS a tree — build this topic and the feature at the same time.

Theory. A binary search tree's O(log n) is conditional on balance — an unbalanced BST (e.g. built by inserting already-sorted data) degenerates into a linked list with O(n) operations, which is precisely why self-balancing variants (AVL, red-black) exist and why Java's TreeMap uses one internally. Tree problems are recursion problems wearing a different hat: almost every traversal, height check, or LCA solution is 'do the recursive case, combine the left and right subtree results.'

Complexity at a Glance

Operation	Time	Space
Traversal (any order)	O(n)	O(n)
BST search / insert / delete	O(log n) balanced / O(n) skewed	
Height / isBalanced check	O(n)	O(n)
Lowest Common Ancestor	O(log n) BST / O(n) general	O(n)

Java Notes. TreeMap/TreeSet are Java's production-grade self-balancing BSTs (red-black trees) — O(log n) guaranteed, unlike a hand-rolled BST.

Interview patterns: DFS (pre/in/post-order) · BFS (level order) · Divide-and-combine on subtrees

Hands-On Projects

Binary Tree Traversals — In-order, pre-order, post-order recursively → level-order (BFS) with a Queue → print all four for the same tree and compare.

Binary Search Tree From Scratch — insert() maintaining BST order → search() in O(log n) on balanced tree → delete() handling leaf/one-child/two-children → in-order iterator confirms sorted output.

Tree Height & Balance Check — Recursive height(node) → isBalanced() checking every subtree's height difference → test balanced and deliberately skewed trees.

Lowest Common Ancestor — LCA for a BST using ordering → LCA for a general binary tree with no ordering → compare approaches → test one node as ancestor of the other.

Serialize & Deserialize a Binary Tree — Design a string format encoding nulls → serialize() → deserialize() rebuilding the exact tree → confirm round-tripping is identical.

SHOPFLOW APPLICATION. This topic is the recursive tree traversal that builds ShopFlow's nested category menu (Electronics → Laptops → Gaming Laptops) — implement it here, wire it into the category API in Part A/C.

Suggested LeetCode volume: 15 problems (4 Easy / 9 Medium / 2 Hard)

Study Resources

- GeeksforGeeks — Tree Data Structure
- Baeldung — Binary Tree in Java
- NeetCode — Trees

9.9 · Heaps & Priority Queues

A tree-shaped structure optimized for one thing: instantly grabbing the smallest (or largest) item.

Subtopics: min-heap vs. max-heap · heapify (sift up/down) · top-K problems · heap sort

PARALLEL WEEK. Week 10, parallel with Part C · Deployment & Cloud (job queues, task scheduling). Any background-job worker you deploy needs a priority queue for retry/urgency ordering — same structure, real system.

Theory. A binary heap is a complete binary tree stored implicitly in an array — no pointers, just index math (children of i are at $2i+1$, $2i+2$) — which is why heapify from an existing array is $O(n)$, not $O(n \log n)$: most nodes near the bottom need almost no sifting. The ‘maintain a heap of size K ’ pattern converts a lot of ‘top- K ’ problems from $O(n \log n)$ full sorts into $O(n \log k)$.

Complexity at a Glance

Operation	Time	Space
insert / extractMin/Max	$O(\log n)$	$O(n)$ total
peek	$O(1)$	—
build heap from array	$O(n)$	$O(n)$
Kth largest via size-K heap	$O(n \log k)$	$O(k)$

Java Notes. PriorityQueue is a min-heap by default; pass a Comparator.reverseOrder() for a max-heap, or a custom comparator for k-closest/task-scheduling style problems.

Interview patterns: Top-K via bounded heap · Two-heap median · Heap + greedy scheduling

Hands-On Projects

Min-Heap From Scratch — insert() with sift-up → extractMin() with sift-down → peek() → confirm heap property after several insert/extract cycles.

Kth Largest Element — Maintain min-heap of size K while scanning → root = Kth largest at the end → compare to fully sorting first → test K = array length.

Heap Sort — Build max-heap from unsorted array → repeatedly extract max, place at end → confirm fully sorted → compare to Java’s built-in sort.

Merge K Sorted Lists — Head of each list into a min-heap → repeatedly extract min, advance that list → build merged result → test very different lengths.

Running Median with Two Heaps — Max-heap for lower half, min-heap for upper half, kept balanced → return median in $O(1)$ → test even and odd counts.

Top-K & Scheduling Patterns — ‘ K Closest Points to Origin’ with max-heap of size K → ‘Task Scheduler’ with max-heap on frequency + cooldown queue → name what both have in common.

SHOPFLOW APPLICATION. This is the priority queue for order-processing-by-urgency/SLA and the min-heap that surfaces lowest-stock items first on the restock dashboard.

Suggested LeetCode volume: 8 problems (2 Easy / 5 Medium / 1 Hard)

Study Resources

- GeeksforGeeks — Heap Data Structure
- Oracle — PriorityQueue (Javadoc)
- NeetCode — Heap / Priority Queue

9.10 · Tries NEW

Promoted from a footnote — it was referenced (‘trie-based autocomplete’) but never actually taught.

Subtopics: TrieNode structure · insert/search/startsWith · deletion · word-frequency & autocomplete ranking

PARALLEL WEEK. Week 10, parallel with Part C · Deployment & Cloud. Build the trie now so the search-bar autocomplete feature has a real data structure behind it before you wire up the deployed search endpoint.

Theory. A trie’s advantage over a HashSet of strings isn’t average-case lookup speed (both are effectively $O(L)$) — it’s that a trie makes prefix queries native: ‘all words starting with “lap”’ is a single subtree walk, whereas a hash set would require scanning every stored word.

Complexity at a Glance

Operation	Time	Space
insert(word), length L	$O(L)$	$O(L)$ worst case
search(word) / startsWith(prefix)	$O(L)$	$O(1)$
Autocomplete: collect all under a prefix	$O(L + \text{results})$	$O(1)$ extra

Java Notes. Back TrieNode.children with a fixed Map<Character, TrieNode> or a 26-slot array for lowercase-only inputs — the array version is faster but only works for a known, bounded alphabet.

Interview patterns: Prefix tree traversal · Trie + backtracking (grid word search)

Hands-On Projects

Trie From Scratch — TrieNode with children map/array + isEndOfWord flag → insert(word) → search(word) exact match → startsWith(prefix).

Delete From a Trie — delete(word) handling the case where the word is a prefix of another → prune childless nodes → test deleting a word sharing a prefix.

Word Search II — Build a Trie from the dictionary → DFS the grid walking the Trie alongside the path → prune as soon as no matching child → test overlapping words.

Autocomplete System — Store frequency/weight at end-of-word nodes → collect completions under a prefix node → sort by frequency then alphabetically → test zero-match prefix.

Longest Common Prefix — Solve via Trie (insert all, walk down while exactly one child) → solve via horizontal string scan → compare for many-short vs. few-long words.

SHOPFLOW APPLICATION. This is the search-bar autocomplete, fully specified: insert every product title into the trie at index time, then startsWith()-walk it on every keystroke.

Suggested LeetCode volume: 5 problems (1 Easy / 3 Medium / 1 Hard)

Study Resources

- GeeksforGeeks — Trie Data Structure in Java
- Baeldung — Trie Data Structure in Java
- NeetCode — Tries

9.11 · Graphs

Nodes and edges modeling relationships — from social networks to delivery routes to the N+1 query problem from Part A.

Subtopics: adjacency list vs. matrix · BFS & DFS · unweighted shortest path (BFS) · topological sort · Dijkstra’s algorithm · union-find

PARALLEL WEEK. Week 11, parallel with Part C · Deployment & Cloud (service dependency graphs, request routing). Microservice call graphs and CDN routing are literally graph problems — this is the week theory and infra conversation overlap most.

Theory. BFS explores in expanding ‘rings’ from the source, which is exactly why it finds shortest paths on unweighted graphs — every edge costs the same, so first-discovered means fewest-edges. Dijkstra is BFS generalized with a priority queue instead of a plain queue, always expanding the currently-cheapest known path instead of the next-in-line one; it breaks on negative edge weights precisely because that cheapest-known assumption stops holding once a later, cheaper path can undercut an already-finalized one.

Complexity at a Glance

Operation	Time	Space
BFS / DFS traversal (V vertices, E edges)	O(V + E)	O(V)
Topological sort (Kahn’s/DFS)	O(V + E)	O(V)
Dijkstra (binary heap)	O((V+E) log V)	O(V)
Union-Find, per op (path compression + rank)	O(α(n)) ≈ O(1)	O(V)

Java Notes. Represent graphs as Map<Node, List> for sparse graphs (most real ones); use an adjacency matrix only when the graph is dense or you need O(1) edge-existence checks.

Interview patterns: BFS shortest path (unweighted) · DFS + backtrack on graphs · Topological ordering · Dijkstra (weighted shortest path)

Hands-On Projects

Graph Representation & Traversal — Adjacency list Map<Node,List> → BFS returning visit order → DFS iteratively (stack) and recursively → test a disconnected component.

Connected Components — Count components in an undirected graph via DFS or union-find → test fully connected vs. fully disconnected.

Unweighted Shortest Path — BFS finds fewest-edges path from source → track distances → reconstruct the actual path → explain why BFS fails once edges have weights (motivates Dijkstra).

Topological Sort — Model tasks with prerequisites as a directed graph → DFS-based or Kahn’s (BFS + in-degree) sort → detect and report a cycle.

Dijkstra’s Shortest Path — Weighted graph (e.g. delivery routes) → priority-queue Dijkstra → shortest distance to every node → test where the ‘obvious’ path isn’t shortest.

Union-Find as an Alternative to DFS — union()/find() with path compression → detect a cycle in an undirected graph → recount connected components, compare to DFS version.

SHOPFLOW APPLICATION. BFS builds the ‘users who bought X also bought Y’ recommendation graph; Dijkstra runs on a city graph for shortest delivery routing — both are on this phase’s final-project list.

Suggested LeetCode volume: 15 problems (3 Easy / 9 Medium / 3 Hard)

Study Resources

- GeeksforGeeks — Graph Data Structure and Algorithms

- Baeldung — Graph Theory in Java
- NeetCode — Graphs

9.12 · Union-Find (Disjoint Set) ELEVATED

Elevated from a sub-item of the Graphs topic to its own short, focused pattern family.

Subtopics: parent array representation · find() with path compression · union() by rank/size · when Union-Find beats DFS

PARALLEL WEEK. Week 11, parallel with Part C · Deployment & Cloud. Same week as Graphs — treat this as a half-day add-on, not a separate multi-day topic.

Theory. Union-Find answers one question extremely fast, repeatedly: ‘are these two things in the same group?’ — where DFS would re-traverse the graph, Union-Find amortizes the cost across every query by flattening the tree structure (path compression) every time it’s touched, converging toward effectively constant time per operation.

Complexity at a Glance

Operation	Time	Space
find() with path compression	$O(\alpha(n)) \approx O(1)$ amortized	$O(n)$
union() by rank/size	$O(\alpha(n)) \approx O(1)$ amortized	$O(n)$
n unions + m finds, total	$O((n+m) \cdot \alpha(n))$	$O(n)$

Java Notes. The inverse Ackermann function $\alpha(n)$ is, for any n you’ll ever encounter, ≤ 4 — treat Union-Find operations as $O(1)$ in interviews without apology.

Interview patterns: Disjoint-set grouping · Cycle detection without DFS · Incremental connectivity

Hands-On Projects

Union-Find From Scratch — find() with path compression → union() by rank or size → confirm two indirectly-unioned elements resolve to the same root.

Number of Provinces / Friend Circles — Union every directly-connected pair → count distinct roots remaining → compare runtime to DFS-based components.

Redundant Connection — Process edges one at a time, unioning each pair → first edge unioning an already-same-set pair is redundant → test a graph with no redundant edge.

Accounts Merge — Union accounts sharing at least one email → group emails by final root account → test transitive chains (A-B, B-C ⇒ A-C merged).

SHOPFLOW APPLICATION. This is the exact algorithm for merging duplicate customer accounts that share an email or phone number — union on shared contact info, then dedupe by root.

Suggested LeetCode volume: 5 problems (1 Easy / 3 Medium / 1 Hard)

Study Resources

- GeeksforGeeks — Disjoint Set (Union-Find)
- NeetCode — Union-Find / Graphs

9.13 · Greedy Algorithms NEW

The conceptual counterweight to DP — teaching DP without ever contrasting it against Greedy leaves a real gap in interview discussions.

Subtopics: the greedy-choice property · optimal substructure · interval scheduling · greedy vs. DP: how to tell which one applies

PARALLEL WEEK. Week 12, parallel with Part C · Deployment & Cloud (resource scheduling, autoscaling policies). Autoscaler ‘add a node when queue depth exceeds X’ logic is greedy — locally optimal, no backtracking.

Theory. A greedy algorithm is only correct when the problem has both the greedy-choice property (a locally optimal choice leads to a globally optimal solution) and optimal substructure — and the interview skill is proving or disproving that, not just writing the loop. 0/1 Knapsack is the canonical counterexample: greedy-by-ratio looks reasonable and is provably wrong, which is exactly why it’s the standard way to teach the boundary between Greedy and DP.

Complexity at a Glance

Operation	Time	Space
Activity/interval selection (sort + scan)	$O(n \log n)$	$O(1)$
Jump Game (single pass)	$O(n)$	$O(1)$
Gas Station (single pass)	$O(n)$	$O(1)$
Huffman coding (min-heap merges)	$O(n \log n)$	$O(n)$

Java Notes. No special API here — the skill is entirely in choosing the right sort key (end time, not start time, for interval scheduling) and proving why that key makes the greedy choice safe.

Interview patterns: Interval scheduling (sort by end time) · Single-pass running-state greedy · Greedy vs. DP boundary reasoning

Hands-On Projects

Activity/Interval Selection — Sort activities by end time → greedily pick next activity starting after the last picked ends → prove in a comment why end-time sort makes the choice correct.

Jump Game — Track farthest reachable index while scanning → false as soon as current index exceeds farthest reachable → extend to Jump Game II (min jumps).

Gas Station — Running tank total across one loop → reset start candidate to next station if total goes negative → prove one linear pass suffices.

Greedy vs. DP: Where Greedy Fails — 0/1 Knapsack via naive greedy (highest value/weight first) → construct a test case where it’s wrong → explain which property breaks vs. Fractional Knapsack.

Huffman Coding (conceptual build) — Frequency map of characters → repeatedly merge two lowest-frequency nodes via min-heap → derive prefix codes → confirm no code prefixes another.

SHOPFLOW APPLICATION. Greedy interval scheduling is the algorithm on this phase’s final-project list for maximizing non-overlapping active promotions/coupons — sort by end date, greedily pack the calendar.

Suggested LeetCode volume: 8 problems (3 Easy / 4 Medium / 1 Hard)

Study Resources

- GeeksforGeeks — Greedy Algorithms
- GeeksforGeeks — Introduction to Greedy Algorithms
- NeetCode — Greedy

9.14 · Bit Manipulation NEW

Low time investment, disproportionately common as an OA screen and ‘easy warm-up’ interview question.

Subtopics: AND/OR/XOR/NOT & shifts · counting set bits · checking powers of two · bitmasking basics

PARALLEL WEEK. Week 12, parallel with Part C · Deployment & Cloud. Half-day add-on alongside Greedy — this topic is intentionally light.

Theory. Bitmasking represents a subset of n items as an n-bit integer — the same subsets you generated with backtracking in 9.7, but here membership is a bit check instead of a recursive branch, trading (in exchange for lower readability) a modest constant-factor speed win and a much smaller memory footprint for storing many subsets at once, e.g. as DP state.

Complexity at a Glance

Operation	Time	Space
Any single bitwise op (AND/OR/XOR/shift)	O(1)	O(1)
Count set bits (Brian Kernighan’s)	O(k) for k set bits	O(1)
Generate all 2^n subsets via bitmask	O(2^n · n)	O(1) extra

Java Notes. Integer.bitCount(n), Integer.numberOfTrailingZeros(n), and Integer.highestOneBit(n) are built-in and O(1)-ish — know they exist before hand-rolling loops for common bit queries.

Interview patterns: XOR cancellation · Bitmask subset representation · Bit-position DP state

Hands-On Projects

Bitwise Operator Drill — setBit/clearBit/toggleBit/checkBit helpers at a given position → verify against a manual binary example → explain why n & (n-1) clears the lowest set bit.

Single Number — XOR every element together → explain why XOR-ing a number with itself cancels to 0 → extend to ‘every element appears three times except one’ via per-position bit counting.

Counting Bits — Baseline with Integer.bitCount() → DP: bits(i) = bits(i>>1) + (i&1) → compare time complexity of both.

Power of Two / Power of Four Check — n > 0 && (n & (n-1)) == 0 for power-of-two → extend to power-of-four accounting for bit position → test 0 and negative input.

Subsets via Bitmask — Represent each subset as an n-bit integer 0..2^n-1 → read included elements by checking each bit → compare to the recursive backtracking version from 9.7.

SHOPFLOW APPLICATION. This is the final-project list’s bitmasking for feature flags & permission sets — store a user’s role/flag combination as a single int and check access with one AND.

Suggested LeetCode volume: 6 problems (4 Easy / 2 Medium)

Study Resources

- GeeksforGeeks — Bitwise Algorithms
- GeeksforGeeks — What is Bit Manipulation
- NeetCode — Bit Manipulation

9.15 · Dynamic Programming

Solving a big problem by breaking it into overlapping smaller subproblems and never solving the same one twice.

Subtopics: memoization vs. tabulation · house-robber-style linear DP · knapsack pattern · longest common subsequence · coin change · state-machine DP

PARALLEL WEEK. Week 13, parallel with Part C wrap-up → Part D full-time. DP is the one topic worth going full-time on — give it a dedicated final week rather than splitting focus with deployment work.

Theory. DP applies exactly when a problem has overlapping subproblems (the same subproblem gets recomputed without caching) and optimal substructure (the optimal solution is built from optimal solutions to subproblems) — recursion + memoization is top-down DP; tabulation is the same recurrence solved bottom-up in a loop, and it’s worth doing both for every problem here because tabulation is what makes the space-optimization step (dropping a 2D table to a rolling 1D array) obvious.

Complexity at a Glance

Operation	Time	Space
1D DP (e.g. House Robber, Coin Change)	$O(n)$ or $O(n \cdot \text{amount})$	$O(n) \rightarrow O(1)$ rolling
2D DP (e.g. 0/1 Knapsack, LCS)	$O(n \cdot m)$	$O(n \cdot m) \rightarrow O(m)$ rolling
Naive recursive Fibonacci (no memo)	$O(2^n)$	$O(n)$ call stack
Memoized Fibonacci	$O(n)$	$O(n)$

Java Notes. A memo cache as `Map<Long, Integer>` or a properly-sized `int[]` — prefer the array when state fits a small bounded range; it avoids autoboxing and hashing overhead entirely.

Interview patterns: Top-down memoization · Bottom-up tabulation · State-machine DP · Space-optimized rolling array

Hands-On Projects

Fibonacci: Naive → Memoized → Tabulated — Naive recursive, timed at $n=35$ → memoization with `HashMap/array` → bottom-up tabulation, no recursion → compare all three.

House Robber — `dp[i] = max take through house i (skip or rob)` → House Robber II with houses in a circle → trace back which houses were chosen.

0/1 Knapsack — 2D DP table (items × capacity) → maximize value → trace back chosen items → test capacity fitting everything vs. almost nothing.

Longest Common Subsequence — 2D DP table comparing two strings → return LCS length → extend to reconstruct the actual subsequence → test strings sharing nothing.

Coin Change (Unbounded Knapsack) — Minimum coins to reach target, return -1 if impossible → count-the-ways variant → compare unbounded vs. 0/1 knapsack’s constraint.

DP on a Grid: Unique Paths — Count unique paths top-left → bottom-right, right/down only → 2D DP table → add obstacles blocking cells → confirm correct counts.

State-Machine DP: Buy/Sell Stock with Cooldown — States: holding / sold-today / free-to-buy → transition equations day by day → extend with a transaction fee → compare to single-transaction stock problem.

SHOPFLOW APPLICATION. 0/1 Knapsack directly generalizes into the backtracking-under-budget bundle generator from 9.7 (the DP version finds the optimal bundle value, not just a valid one) — build both and compare which ShopFlow actually needs.

Suggested LeetCode volume: 20 problems (3 Easy / 12 Medium / 5 Hard)

Study Resources

- [GeeksforGeeks](#) — Dynamic Programming
- [Baeldung](#) — Dynamic Programming in Java
- [NeetCode](#) — Dynamic Programming

9.16 · Segment Trees & Fenwick Trees OPTIONAL

Most companies don’t ask this outside HFT/quant or competitive-programming-heavy bars. Include only if that’s your target.

Subtopics: segment tree build/query/update · lazy propagation (concept only) · Fenwick Tree (Binary Indexed Tree) for prefix sums

PARALLEL WEEK. Optional — outside the parallel schedule. Only revisit this after Part E if you’re specifically targeting quant/HFT or competitive-programming-heavy interview loops.

Theory. Both structures answer ‘range query with updates’ faster than the $O(n)$ recompute a plain prefix-sum array requires after every mutation — the Fenwick Tree trades generality for a smaller, faster constant factor via clever bit-index tricks, while segment trees generalize to range min/max/GCD, not just sums.

Complexity at a Glance

Operation	Time	Space
Segment tree build	$O(n)$	$O(n)$
Segment tree range query / point update	$O(\log n)$	$O(n)$

Fenwick Tree update / prefixSum	O(log n)	O(n)
---------------------------------	----------	------

Java Notes. No standard-library equivalent exists — production code typically reaches for a database index or a specialized library before hand-rolling either structure.

Interview patterns: Range query + point update · Bit-trick indexing (Fenwick)

Hands-On Projects

Segment Tree for Range Sum Queries — Build in O(n) → range-sum query in O(log n) → point update in O(log n) → compare to Prefix Sum from 9.1, explain when each is right.

Fenwick Tree (Binary Indexed Tree) — update() and prefixSum() via BIT bit-trick indexing → compare code complexity/constant factor to segment tree → test both on the same workload.

SHOPFLOW APPLICATION. Not part of the core ShopFlow build — relevant only if you extend into a high-throughput analytics/leaderboard feature needing sub-millisecond range updates at scale.

Suggested LeetCode volume: 4 problems (optional — only if targeting quant/HFT/competitive programming)

Study Resources

- GeeksforGeeks — Segment Tree
- GeeksforGeeks — Binary Indexed Tree

Final Project for This Phase — DSA Inside ShopFlow

Every topic above already ends in a ShopFlow Application callout — this table is the consolidated feature list so the whole phase’s payoff is visible in one place. Build these into the actual running ShopFlow codebase, not as a separate throwaway exercise.

ShopFlow Feature	DSA Technique	Topic
Product search autocomplete	Trie-based prefix search	9.10
Nested category menu	Recursive tree traversal	9.0 / 9.8
Order processing by urgency/SLA	Priority queue (heap)	9.9
“Users who bought X also bought Y”	Graph traversal (BFS)	9.11
Restock dashboard, lowest-stock first	Min-heap	9.9
Delivery routing	Dijkstra on a city graph	9.11
Coupon/promotion scheduling	Greedy interval scheduling	9.1 / 9.13
Duplicate customer merge	Union-Find on shared email/phone	9.12
Bundle/combo offer generator	Backtracking under a price constraint	9.7 / 9.15
Feature flags & permission sets	Bitmasking	9.14
Recently-viewed products rail	LRU cache (HashMap + doubly linked list)	9.6
Cursor pagination / infinite scroll	Sliding window	9.3

Part D (Integrated Edition) complete. DSA is no longer a block that comes after the app is built — it ran alongside Parts A-C the whole way, and every structure above already lives inside ShopFlow. Next: Part E — Career & Interview Readiness.

Progress Tracker

Topic	Week	Projects	LC
9.0 Recursion & Recursive Thinking (NEW)	1-2	5	8
9.1 Arrays & Strings	1-3	6	15
9.2 Hashing	3	5	10
9.3 Two Pointers & Sliding Window — Pattern Review (NEW)	4	5	8
9.4 Stacks & Queues	5	6	10
9.5 Sorting & Searching	6	5	10
9.6 Linked Lists	7	6	10
9.7 Backtracking (NEW)	8	5	10
9.8 Trees	9	5	15
9.9 Heaps & Priority Queues	10	6	8
9.10 Tries (NEW)	10	5	5

9.11 Graphs	11	6	15
9.12 Union-Find (Disjoint Set) (ELEVATED)	11	4	5
9.13 Greedy Algorithms (NEW)	12	5	8
9.14 Bit Manipulation (NEW)	12	5	6
9.15 Dynamic Programming	13	7	20
9.16 Segment Trees & Fenwick Trees (OPTIONAL / ADVANCED)	Optional — outside the parallel schedule	2	4

Total: 17 topics (16 core + 1 optional) · 85 hands-on projects · ~163 suggested LeetCode problems · 13 weeks, run concurrently with Parts A-C.